

1 Manajemen Service

Fokus Bab

Pahami alur hidup service Linux modern, dari unit systemd dan ketergantungan hingga logging dan hardening service jaringan.



systemd



unit



journal

Server yang kamu kelola tidak berhenti bekerja saat terminal ditutup. Ada program yang terus berjalan di latar belakang: menerima koneksi SSH, melayani permintaan web, mencatat aktivitas sistem ke log. Program-program ini disebut *service*, yaitu proses yang hidup terus sampai dihentikan secara eksplisit atau sistem dimatikan.

Koneksi SSH yang kamu buka ke server diproses oleh *service* bernama *sshd* yang sudah mendengarkan di port 22 jauh sebelum kamu terhubung. Halaman web yang muncul di browser berasal dari *service* web server yang siap menjawab sejak sistem menyala. Service semacam ini dikelola oleh *init system*. Pada Ubuntu 22.04 dan distribusi Linux modern lainnya, *init system* yang digunakan adalah **systemd**. Mengelola *service* adalah salah satu tugas paling dasar seorang administrator sistem. Memulai, menghentikan, mengaktifkan saat boot, memeriksa status, membaca log ketika ada masalah: semua itu dilakukan melalui satu perintah utama bernama *systemctl*. Bab ini membahas cara menguasai perintah itu dan memahami struktur di baliknya.

Yang Akan Kamu Pelajari

Setelah menyelesaikan bab ini, kamu akan mampu:

1. menjelaskan evolusi *init system* di Linux dan peran *systemd* sebagai PID 1;
2. mengelola siklus hidup layanan menggunakan *systemctl* dengan tepat;
3. membuat dan memodifikasi berkas unit *systemd* untuk layanan kustom;
4. memfilter dan menganalisis log layanan menggunakan *journalctl*;
5. mengonfigurasi layanan SSH sebagai contoh nyata konfigurasi layanan jaringan; serta
6. melakukan troubleshooting layanan yang gagal start dengan urutan diagnostik yang terstruktur.



Catatan

Seluruh praktek pada bab ini dijalankan pada direktori `/lab-os/chapter10-services/`.

```
mkdir -p ~/lab-os/chapter10-services
cd ~/lab-os/chapter10-services
```

1.1 Pengenalan systemd

Setelah kernel Linux selesai memuat dirinya sendiri, ia menjalankan proses pertama di *user space*. Proses ini mendapat nomor proses (PID) 1 dan bertanggung jawab memulai semua layanan berikutnya: jaringan, logging, printer, server web, database, dan seterusnya. Proses dengan PID 1 inilah yang disebut *init system*.

Secara historis, Linux menggunakan **SysV init** yang berbasis skrip shell berurutan di direktori `/etc/init.d/`. Setiap layanan diwakili oleh sebuah skrip yang bisa dipanggil dengan argumen *start*, *stop*, atau *restart*. Pendekatannya sederhana dan mudah dipahami, tetapi lambat karena layanan dijalankan satu per satu secara berurutan.

Ubuntu pernah menggunakan **Upstart** sebagai transisi, yang memperkenalkan pendekatan berbasis

event. Upstart lebih fleksibel dari SysV, tetapi adopsinya tidak meluas ke seluruh ekosistem Linux. Hari ini, hampir semua distribusi Linux modern, termasuk Ubuntu 22.04, Debian 12, Fedora, RHEL, dan turunannya, menggunakan **systemd**. Systemd memperkenalkan beberapa kemajuan penting: layanan dijalankan secara paralel selama boot, dependensi antar layanan dinyatakan secara eksplisit, logging terintegrasi langsung ke dalam sistem, dan semua unit dikelola melalui antarmuka yang seragam.

```
# periksa PID 1
ps -p 1 -o pid,comm,args=
# output: 1 systemd /sbin/init (atau /lib/systemd/systemd)

# verifikasi versi systemd
systemctl --version

# analisis waktu boot layanan -- berapa lama tiap layanan butuh start
systemd-analyze
systemd-analyze blame
systemd-analyze blame | head -20

# lihat rantai kritis boot
systemd-analyze critical-chain
```

Kode 1.1: Verifikasi init system dan analisis waktu boot

Dalam systemd, semua yang dikelola disebut *unit*. Ada beberapa tipe *unit*, tetapi yang paling sering ditemui adalah:

- *.service*: layanan yang berjalan sebagai proses latar belakang
- *.socket*: *endpoint* jaringan atau IPC; layanan terkait baru dijalankan saat ada koneksi masuk
- *.timer*: tugas terjadwal, alternatif modern untuk cron
- *.target*: pengelompokan unit, menggantikan konsep *runlevel* lama
- *.mount*: titik *mount filesystem* yang dikelola systemd

Berkas unit disimpan di beberapa lokasi dengan prioritas berbeda:

- */lib/systemd/system/* atau */usr/lib/systemd/system/*: unit bawaan paket, jangan diedit langsung
- */etc/systemd/system/*: unit kustom dan *override* lokal, tempat kamu bekerja
- */.config/systemd/user/*: unit tingkat pengguna, tidak perlu hak root

Praktek 10.1: Amati Layanan Aktif Saat Boot

Periksa layanan yang berjalan di sistem dan identifikasi layanan yang paling lambat saat boot.

1. Lihat semua layanan yang sedang berjalan.

```
systemctl list-units --type=service --state=running
# catat berapa banyak layanan yang aktif
```

Kode 1.2: Melihat layanan aktif

Amati: Setiap baris menampilkan nama unit, status aktif/tidaknya, status dari sudut pandang systemd, dan deskripsi singkat.

2. Lihat semua unit service yang ada (aktif maupun tidak).

```
systemctl list-unit-files --type=service | head -30
# enabled = akan start otomatis saat boot
# disabled = tidak start otomatis, bisa dijalankan manual
```

```
# static = tidak bisa di-enable/disable, hanya dipanggil oleh layanan lain
```

Kode 1.3: Melihat Semua Unit Service

3. Analisis waktu boot dan temukan layanan paling lambat.

```
systemd-analyze  
systemd-analyze blame | head -15
```

Kode 1.4: Analisis waktu boot

Amati: Output `systemd-analyze blame` menampilkan daftar unit yang diurutkan dari yang paling lama hingga paling cepat inisialisasinya.

➤ Tantangan

Identifikasi tiga layanan dengan waktu inisialisasi terlama menggunakan `systemd-analyze blame`. Gunakan pipeline dari Bab 3 (`| sort -rh | head -3`) untuk mempercepat pencariannya. Untuk setiap layanan, cari tahu fungsinya dengan `systemctl cat nama-layanan`. Tuliskan nama layanan, waktu inisialisasinya, dan penjelasan singkat fungsinya.

1.2 Mengelola Layanan dengan systemctl

`systemctl` adalah satu-satunya perintah yang perlu kamu kuasai untuk mengelola layanan di `systemd`. Perintah ini menggantikan berbagai perintah lama seperti `service`, `chkconfig`, dan `update-rc.d` yang dulu tersebar di berbagai distribusi.

Ada dua dimensi dalam mengelola layanan: *status runtime* (apakah layanan sedang berjalan saat ini) dan *status boot* (apakah layanan akan berjalan otomatis setiap kali sistem menyala). Keduanya dikontrol secara terpisah.

```
# periksa status lengkap layanan  
sudo systemctl status ssh  
  
# mulai layanan yang sedang mati  
sudo systemctl start ssh  
  
# hentikan layanan  
sudo systemctl stop ssh  
  
# restart: hentikan lalu jalankan ulang  
sudo systemctl restart ssh  
  
# reload: muat ulang konfigurasi tanpa mematikan proses utama  
sudo systemctl reload ssh  
  
# periksa apakah layanan sedang berjalan (untuk skrip)  
systemctl is-active ssh  
# output: active atau inactive  
  
# periksa apakah layanan akan start saat boot  
systemctl is-enabled ssh  
# output: enabled atau disabled
```

Kode 1.5: Perintah dasar systemctl untuk runtime layanan

Perbedaan antara `restart` dan `reload` sangat penting di lingkungan produksi. `restart` mematikan proses lama sepenuhnya lalu menjalankan yang baru: semua koneksi aktif terputus. `reload` meminta

proses yang sedang berjalan untuk membaca ulang konfigurasinya tanpa mati: koneksi aktif tetap terjaga. Tidak semua layanan mendukung reload; cek dokumentasinya lebih dulu.

Untuk mengontrol apakah layanan berjalan otomatis saat boot, gunakan enable dan disable:

```
# aktifkan layanan agar start otomatis saat boot
sudo systemctl enable ssh

# aktifkan sekaligus jalankan sekarang juga
sudo systemctl enable --now ssh

# nonaktifkan auto-start (layanan masih bisa dijalankan manual)
sudo systemctl disable ssh

# nonaktifkan sekaligus hentikan sekarang
sudo systemctl disable --now ssh

# blokir total layanan -- tidak bisa dijalankan manual maupun otomatis
sudo systemctl mask cups
# untuk membuka blokir:
sudo systemctl unmask cups
```

Kode 1.6: Mengontrol auto-start layanan saat boot

Perintah mask lebih kuat dari disable. Ketika layanan di-mask, systemd mengarahkan berkas unitnya ke /dev/null, sehingga tidak bisa dijalankan dengan cara apapun sampai di-unmask. Ini berguna untuk menonaktifkan layanan yang tidak ingin pernah aktif, misalnya layanan printer di server yang tidak memiliki printer.

Perintah `systemctl list-units` membantu melihat gambaran besar layanan di sistem:

```
# semua unit service yang sedang berjalan
systemctl list-units --type=service --state=running

# semua unit yang gagal
systemctl --failed

# semua unit service dan status boot mereka
systemctl list-unit-files --type=service

# lihat dependensi sebuah layanan
systemctl list-dependencies ssh
```

Kode 1.7: Melihat dan memfilter daftar unit

Praktek 10.2: Kelola Layanan SSH

Praktikkan semua operasi dasar systemctl pada layanan SSH yang tersedia di sistem.

1. Periksa status SSH secara menyeluruh.

```
systemctl status ssh
systemctl is-active ssh
systemctl is-enabled ssh
```

Kode 1.8: Memeriksa status layanan SSH

Amati: Output `systemctl status` menampilkan banyak informasi: status aktif/inactive, PID proses utama, waktu mulai, penggunaan memori, dan beberapa baris log terbaru. Ini adalah perintah pertama yang harus dijalankan ketika ada masalah layanan.

2. Lakukan restart dan pantau perubahannya.

```
sudo systemctl restart ssh
systemctl status ssh
# perhatikan: Loaded, Active, dan Main PID bisa berubah setelah restart
```

Kode 1.9: Restart layanan dan pantau status

3. Lihat dependensi SSH.

```
systemctl list-dependencies ssh
# layanan lain yang harus aktif sebelum SSH bisa berjalan
```

Kode 1.10: Melihat dependensi layanan SSH

4. Cek semua unit yang gagal di sistem.

```
systemctl --failed
# jika ada, ini adalah daftar layanan yang butuh perhatian
```

Kode 1.11: Audit layanan yang gagal

➤ Tantangan

Buat skrip Bash (referensi Bab 7) bernama `cek-layanan.sh` yang memeriksa status daftar layanan dari sebuah berkas teks. Berkas teks `daftar-layanan.txt` berisi satu nama layanan per baris (isi minimal: `ssh`, `cron`, `rsyslog`). Skrip membaca setiap nama layanan, memeriksa statusnya dengan `systemctl is-active`, lalu menulis laporan ke berkas `laporan-layanan.log` dengan format: `[TANGGAL] nama-layanan: ACTIVE/INACTIVE`. Gunakan `date` untuk mendapatkan tanggal.

1.3 Membuat Berkas Unit Kustom

Setiap layanan di `systemd` direpresentasikan oleh sebuah *berkas unit* dengan ekstensi `.service`. Berkas ini adalah deklarasi: apa yang dijalankan, bagaimana menjalankannya, kapan layanan ini harus mulai, dan apa yang harus dilakukan jika layanan crash.

Sebuah berkas unit service memiliki tiga blok utama yang diapit tanda kurung siku:

```
[Unit]
Description=Nama Deskriptif Layanan
Documentation=man:nama-program(8)
After=network.target
Wants=network-online.target

[Service]
Type=simple
User=www-data
Group=www-data
WorkingDirectory=/opt/aplikasi
ExecStart=/usr/local/bin/aplikasi --port 8080
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=5s
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Kode 1.12: Struktur berkas unit `.service`

Blok [Unit] berisi metadata dan relasi dengan unit lain. Description adalah nama yang muncul di output `systemctl status`. After menyatakan urutan start: unit ini baru dijalankan setelah unit yang disebutkan sudah aktif. Wants adalah relasi lemah: `systemd` akan mencoba menjalankan unit tersebut, tetapi kegagalannya tidak menghentikan layanan ini.

Blok [Service] mendefinisikan cara proses dijalankan. ExecStart adalah perintah utama. User dan Group menentukan identitas proses. Restart=on-failure membuat `systemd` secara otomatis menjalankan ulang layanan jika prosesnya berhenti dengan status error.

Blok [Install] menentukan bagaimana unit dihubungkan ke target boot. WantedBy=multi-user.target berarti layanan ini akan diaktifkan sebagai bagian dari target multi-user, yang adalah mode normal operasi server Linux.

Nilai Type pada blok [Service] menentukan bagaimana `systemd` tahu kapan layanan dianggap sudah "berjalan":

- simple: proses pada ExecStart adalah proses utama. Paling sering dipakai.
- forking: program pada ExecStart melakukan fork dan induknya berhenti. Dipakai oleh daemon klasik.
- oneshot: proses berjalan sekali lalu berhenti. Dipakai untuk skrip inisialisasi.
- notify: proses mengirim notifikasi ke `systemd` saat sudah siap menerima koneksi.

Praktek 10.3: Buat Layanan Sederhana dari Skrip Bash

Buat layanan `systemd` yang menjalankan server HTTP sederhana berbasis Python.

1. Siapkan konten yang akan dilayani.

```
cd ~/lab-os/chapter10-services
mkdir -p situs-demo
nano situs-demo/index.html
# Tulis isi berkas berikut
<!DOCTYPE html>
<html>
<body>
<h1>Halo dari layanan systemd kustom!</h1>
<p>Layanan ini dibuat pada praktek Bab 10.</p>
</body>
</html>
```

Kode 1.13: Menyiapkan direktori dan konten layanan

2. Buat skrip wrapper untuk server HTTP.

```
nano ~/lab-os/chapter10-services/jalankan-server.sh
# Tulis isi berkas berikut
#!/bin/bash
DIREKTORI="$HOME/lab-os/chapter10-services/situs-demo"
PORT=9090
echo "Memulai server di port $PORT..."
exec python3 -m http.server $PORT --directory "$DIREKTORI"
chmod +x ~/lab-os/chapter10-services/jalankan-server.sh
```

Kode 1.14: Skrip server HTTP untuk layanan

3. Buat berkas unit `systemd` untuk layanan ini.

```
nano ~/lab-os/chapter10-services/demo-web.service
# Tulis isi berkas berikut
[Unit]
```

```

Description=Demo Web Server Praktek Bab 10
After=network.target

[Service]
Type=simple
User=nama-pengguna-kamu
WorkingDirectory=/home/nama-pengguna-kamu/lab-os/chapter10-services/
    situs-demo
ExecStart=/usr/bin/python3 -m http.server 9090
Restart=on-failure
RestartSec=3s

[Install]
WantedBy=multi-user.target

# salin ke lokasi unit systemd
sudo cp ~/lab-os/chapter10-services/demo-web.service /etc/systemd/
    system/demo-web.service
# minta systemd membaca ulang berkas unit yang baru dibuat
sudo systemctl daemon-reload

```

Kode 1.15: Membuat berkas unit kustom

Amati: Perintah `daemon-reload` tidak me-restart layanan yang sedang berjalan. Perintah ini hanya memberi tahu systemd untuk membaca ulang definisi unit dari disk. Harus selalu dijalankan setiap kali berkas unit diubah.

4. Jalankan layanan dan verifikasi.

```

sudo systemctl start demo-web
systemctl status demo-web
# coba akses layanan
curl http://localhost:9090

```

Kode 1.16: Menjalankan dan memverifikasi layanan kustom

5. Uji fitur restart otomatis.

```

# lihat PID proses saat ini
systemctl status demo-web | grep "Main PID"

# hentikan proses secara paksa (simulasi crash)
sudo kill -9 $(systemctl show demo-web --property=MainPID --value)

# tunggu beberapa detik lalu cek -- systemd harus menghidupkannya
    kembali
sleep 5
systemctl status demo-web
# PID akan berubah karena proses baru dijalankan

```

Kode 1.17: Menguji restart otomatis

Amati: Berkat `Restart=on-failure`, systemd secara otomatis menjalankan ulang layanan setelah proses mati secara tidak normal. Ini adalah fitur penting untuk layanan produksi.

6. Bersihkan layanan uji setelah selesai.

```

sudo systemctl disable --now demo-web
sudo rm /etc/systemd/system/demo-web.service
sudo systemctl daemon-reload

```

Kode 1.18: Membersihkan layanan uji

Tantangan

Modifikasi berkas unit `demo-web.service` sebelum menghapusnya: tambahkan `RestartSec=10s` agar sistem menunggu 10 detik sebelum mencoba restart, dan tambahkan `Environment="PORT=9091"` lalu ubah `ExecStart` agar menggunakan variabel tersebut. Aktifkan layanan dengan `enable` dan `WantedBy=multi-user.target`, lalu uji apakah layanan aktif setelah `systemctl daemon-reload`. Dokumentasikan perbedaan perilaku dibanding versi sebelumnya.

1.4 Logging dengan journalctl

Pada sistem dengan SysV init, log layanan tersebar di berbagai berkas di `/var/log/`: `syslog`, `auth.log`, `kern.log`, dan berkas khusus tiap aplikasi. Mencari satu peristiwa yang melibatkan beberapa layanan berarti membuka banyak berkas sekaligus. Systemd menggabungkan semua log ke dalam *journal* yang dikelola oleh `systemd-journald`. Satu perintah, `journalctl`, bisa mengakses semua log itu dengan berbagai filter.

Log journal disimpan secara biner (bukan teks biasa), sehingga lebih efisien dan mendukung berbagai metadata seperti nama unit, PID, UID, prioritas, dan timestamp presisi tinggi. Ketika menjalankan `journalctl`, ia membaca dan menampilkan log dalam format teks yang mudah dibaca.

```
# semua log sejak boot saat ini
journalctl -b

# log untuk unit/layanan tertentu
journalctl -u ssh
journalctl -u ssh -b
# -b membatasi ke log sejak boot saat ini saja

# N baris terakhir
journalctl -u ssh -n 50

# ikuti log secara real-time (seperti tail -f)
journalctl -u ssh -f

# filter berdasarkan waktu
journalctl -u ssh --since "1 hour ago"
journalctl -u nginx --since "2024-01-15 08:00" --until "2024-01-15 10:00"
journalctl -u ssh --since today

# filter berdasarkan prioritas
# 0=emerg, 1=alert, 2=crit, 3=err, 4=warning, 5=notice, 6=info, 7=debug
journalctl -p err
journalctl -u ssh -p warning

# tampilkan tanpa pager (berguna untuk pipeline)
journalctl -u ssh --no-pager | grep "Failed"
```

Kode 1.19: Perintah-perintah dasar journalctl

Secara default, journal disimpan hanya di memori dan hilang saat sistem reboot. Untuk menyimpan

log secara permanen di disk:

```
# buat direktori penyimpanan journal permanen
sudo mkdir -p /var/log/journal
sudo systemd-tmpfiles --create --prefix /var/log/journal

# restart journald untuk menerapkan perubahan
sudo systemctl restart systemd-journal

# verifikasi -- sekarang ada direktori dengan nama mesin
ls /var/log/journal/
```

Kode 1.20: Mengaktifkan persistent logging

Ukuran journal bisa dibatasi agar tidak memenuhi disk. Konfigurasi ada di `/etc/systemd/journald.conf`:

```
# lihat berapa banyak ruang yang dipakai journal
journalctl --disk-usage

# hapus log lama (pertahankan 1 minggu saja)
sudo journalctl --vacuum-time=7d

# hapus log lama (pertahankan maksimal 500 MB)
sudo journalctl --vacuum-size=500M
```

Kode 1.21: Memeriksa penggunaan dan batas journal

Praktek 10.4: Filter dan Analisis Log Layanan

Gunakan berbagai opsi `journalctl` untuk mencari informasi spesifik dalam log sistem.

1. Lihat log SSH dari satu jam terakhir.

```
journalctl -u ssh --since "1 hour ago" --no-pager
# jika tidak ada log SSH, gunakan layanan lain yang aktif
journalctl -u cron --since "1 hour ago" --no-pager
```

Kode 1.22: Log SSH satu jam terakhir

2. Filter log berprioritas error ke atas.

```
journalctl -b -p err --no-pager
# ini menampilkan semua error dan yang lebih serius sejak boot
```

Kode 1.23: Log prioritas error dari boot saat ini

Amati: Log dengan prioritas `err` ke atas biasanya perlu diperhatikan. Catat layanan mana yang menghasilkan error dan bacalah pesannya.

3. Ikuti log secara real-time sambil memicu aktivitas.

```
# jalankan di terminal pertama:
journalctl -u ssh -f

# di terminal kedua, coba login SSH ke localhost
# ssh localhost
# lalu lihat apa yang muncul di terminal pertama
```

Kode 1.24: Pantau log SSH secara real-time

4. Ekstrak log ke berkas untuk analisis.

```
cd ~/lab-os/chapter10-services

# simpan semua log layanan ssh dari hari ini ke berkas
```

```
journalctl -u ssh --since today --no-pager > log-ssh-hari-ini.txt

# hitung jumlah baris log
wc -l log-ssh-hari-ini.txt

# jika ada, cari baris yang mengandung kata "error" atau "failed"
grep -i "error\|failed" log-ssh-hari-ini.txt | head -20
```

Kode 1.25: Simpan log ke berkas

➤ Tantangan

Ekstrak semua log dengan prioritas error (-p err) dari 24 jam terakhir untuk layanan SSH, simpan ke berkas error-ssh-24jam.txt. Gunakan pipeline dari Bab 3 untuk menghitung total jumlah baris error dengan wc -l, lalu tampilkan 10 pesan error yang paling sering muncul menggunakan sort | uniq -c | sort -rn | head -10. Tuliskan perintah lengkap yang kamu gunakan.

1.5 Konfigurasi Layanan Jaringan

Layanan jaringan adalah kategori layanan yang mendengarkan koneksi masuk melalui port tertentu. SSH mendengarkan di port 22, HTTP di port 80, HTTPS di port 443. Mengelola layanan jaringan berarti memastikan proses berjalan, port terbuka dan mendengarkan, konfigurasi sesuai kebutuhan, dan log dipantau untuk mendeteksi akses tidak sah.

SSH (*Secure Shell*) adalah layanan paling fundamental di server Linux. Ia memungkinkan administrator mengelola server dari jarak jauh melalui koneksi terenkripsi. Paket yang menyediakan SSH server di Ubuntu adalah openssh-server. Nama unit systemd-nya bisa ssh atau sshd tergantung distribusi.

```
# pastikan SSH server terinstal
sudo apt install -y openssh-server

# periksa status
systemctl status ssh

# pastikan SSH aktif dan mulai otomatis saat boot
sudo systemctl enable --now ssh

# verifikasi port 22 mendengarkan
ss -tlnp | grep :22
# atau
netstat -tlnp | grep :22
```

Kode 1.26: Mengelola layanan SSH

Konfigurasi utama SSH server berada di /etc/ssh/sshd_config. Sebelum mengubah apapun, penting memahami beberapa opsi kunci:

```
# lihat konfigurasi aktif (bukan komentar)
grep -v "^#" /etc/ssh/sshd_config | grep -v "^$"

# opsi yang sering dikonfigurasi:
# Port 22 -- port yang didengarkan (ubah untuk keamanan)
# PermitRootLogin no -- larang login langsung sebagai root
# PasswordAuthentication yes -- izinkan login dengan password (ubah ke no
    untuk keamanan lebih tinggi)
# PubkeyAuthentication yes -- izinkan login dengan kunci publik
```

```
# MaxAuthTries 3          -- maksimum percobaan login sebelum koneksi
                           ditolak
# AllowUsers alice bob    -- hanya pengguna tertentu yang boleh SSH
```

Kode 1.27: Opsi penting di sshd_config

Setelah mengubah konfigurasi SSH, selalu validasi sintaks sebelum me-restart layanan. Kesalahan konfigurasi bisa mengunci akses remote ke server:

```
# 1. buat backup konfigurasi sebelum mengubah
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.backup

# 2. ubah konfigurasi dengan editor
sudo nano /etc/ssh/sshd_config

# 3. validasi sintaks -- JANGAN skip langkah ini
sudo sshd -t
# jika tidak ada output, sintaks benar
# jika ada pesan error, perbaiki dulu sebelum lanjut

# 4. muat ulang konfigurasi (lebih aman dari restart)
sudo systemctl reload ssh
# atau restart jika diperlukan
sudo systemctl restart ssh

# 5. verifikasi layanan masih berjalan
systemctl status ssh
```

Kode 1.28: Alur aman mengubah konfigurasi SSH

Gunakan ss (dari Bab 2) untuk memverifikasi bahwa layanan benar-benar mendengarkan di port yang diinginkan:

```
# lihat semua port TCP yang mendengarkan
ss -tlnp
# -t: TCP saja
# -l: hanya yang sedang listen (mendengarkan)
# -n: tampilkan angka (jangan resolve nama)
# -p: tampilkan proses yang menggunakan socket

# cari port SSH secara spesifik
ss -tlnp | grep ssh
```

Kode 1.29: Verifikasi port dengan ss

Praktek 10.5: Konfigurasi SSH Server

Ubah port SSH ke port non-standar, validasi konfigurasi, dan verifikasi perubahan dengan ss.

1. Periksa konfigurasi SSH saat ini.

```
sudo grep -n "^Port|^#Port" /etc/ssh/sshd_config
ss -tlnp | grep ssh
```

Kode 1.30: Periksa konfigurasi SSH awal

2. Buat backup dan ubah port SSH.

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.backup.lab12

# ubah port dari 22 ke 2222 (atau port lain yang belum dipakai)
sudo sed -i 's/^#Port 22/Port 2222/' /etc/ssh/sshd_config
```

```
# jika baris Port 22 tidak dikomentari:
# sudo sed -i 's/^Port 22/Port 2222/' /etc/ssh/sshd_config

# verifikasi perubahan
grep "^Port" /etc/ssh/sshd_config
```

Kode 1.31: Backup dan ubah port SSH

3. Validasi konfigurasi dan restart layanan.

```
# WAJIB: validasi sintaks sebelum restart
sudo sshd -t
echo "Kode keluar sshd -t: $?"
# kode 0 berarti sintaks valid

# restart layanan
sudo systemctl restart ssh
systemctl status ssh
```

Kode 1.32: Validasi dan restart SSH

Amati: Langkah validasi dengan `sshd -t` adalah kebiasaan penting. Pada server produksi yang hanya bisa diakses lewat SSH, kesalahan konfigurasi yang menyebabkan SSH tidak bisa restart berarti kamu terkunci dari server sendiri.

4. Verifikasi port baru dengan ss.

```
ss -tlnp | grep ssh
# seharusnya menampilkan port 2222, bukan 22

# simpan hasil ke berkas bukti
ss -tlnp | grep ssh > ~/lab-os/chapter10-services/bukti-port-ssh.txt
cat ~/lab-os/chapter10-services/bukti-port-ssh.txt
```

Kode 1.33: Verifikasi port SSH baru

5. Kembalikan port SSH ke 22 setelah praktek.

```
sudo cp /etc/ssh/sshd_config.backup.lab12 /etc/ssh/sshd_config
sudo sshd -t
sudo systemctl restart ssh
ss -tlnp | grep ssh
# harus kembali ke port 22
```

Kode 1.34: Kembalikan konfigurasi SSH ke port 22

➤ Tantangan

Ubah konfigurasi SSH untuk menambahkan dua pengaturan keamanan: `PermitRootLogin no` (larang login root langsung) dan `MaxAuthTries 3` (maksimal tiga kali percobaan). Lakukan dengan urutan yang aman: backup, edit, validasi dengan `sshd -t`, reload. Verifikasi perubahan dengan `grep -E "PermitRoot|MaxAuth" /etc/ssh/sshd_config`. Kemudian periksa log SSH untuk memastikan tidak ada error setelah perubahan dengan `journalctl -u ssh -n 20`. Referensi Bab 2 untuk penggunaan `ss` dan Bab 9 untuk keamanan pengguna.

1.6 Rangkuman

Dalam bab ini, kamu telah mempelajari:

- **Evolusi init system:** Linux berevolusi dari SysV init berbasis skrip berurutan, melalui Upstart

berbasis event, ke systemd modern yang menjalankan layanan secara paralel dengan manajemen dependensi eksplisit.

- **Tipe unit systemd:** .service untuk proses latar belakang, .socket untuk endpoint jaringan, .timer untuk tugas terjadwal, .target untuk pengelompokan.
- **systemctl runtime:** start, stop, restart, reload, dan status untuk mengontrol layanan yang sedang berjalan; enable/disable untuk auto-start saat boot; mask/unmask untuk blokir total.
- **Berkas unit kustom:** struktur [Unit], [Service], [Install]; field penting seperti ExecStart, Restart=on-failure, RestartSec, User; selalu jalankan daemon-reload setelah mengubah berkas unit.
- **journalctl:** filter log per unit (-u), per waktu (-since), per prioritas (-p), mode follow (-f), dan ekspor ke berkas dengan -no-pager.
- **Konfigurasi SSH:** berkas konfigurasi di /etc/ssh/sshd_config; alur aman ubah konfigurasi: backup, edit, validasi dengan sshd -t, lalu reload; verifikasi port aktif dengan ss -tlnp.

1.7 Latihan

Instruksi Umum: Kerjakan seluruh latihan secara mandiri. Catat langkah penting, simpan tangkapan layar bila diperlukan, lalu rangkum hasilnya sebagai dokumentasi pribadi.

Latihan 10.1 Audit Layanan dan Analisis Boot

Lakukan audit menyeluruh terhadap layanan yang berjalan di sistem.

1. Jalankan `systemctl list-units -type=service -state=running` dan catat semua layanan aktif. Pilih tiga layanan yang kamu kenal, periksa status masing-masing dengan `systemctl status`, dan jelaskan fungsinya.
2. Jalankan `systemd-analyze blame` dan identifikasi lima layanan dengan waktu inisialisasi terlama. Tampilkan hasilnya menggunakan pipeline: `systemd-analyze blame | head -5`.
3. Jalankan `systemctl -failed` dan dokumentasikan hasilnya. Jika ada layanan yang gagal, cari tahu penyebabnya dengan `journalctl -u nama-layanan -n 30`.

Latihan 10.2 Layanan Kustom dengan Restart Otomatis

Buat layanan systemd kustom yang mendemonstrasikan fitur restart otomatis.

1. Buat skrip Bash (referensi Bab 7) bernama `monitor-disk.sh` yang setiap 30 detik menuliskan penggunaan disk ke berkas log. Gunakan `df -h` dan `date`.
2. Buat berkas unit `/etc/systemd/system/monitor-disk.service` untuk menjalankan skrip tersebut dengan konfigurasi: `Restart=always`, `RestartSec=5s`, dan berjalan sebagai pengguna kamu sendiri.
3. Aktifkan dan jalankan layanan. Verifikasi dengan `systemctl status` dan pastikan log masuk ke journal.
4. Simulasikan crash dengan membunuh proses secara paksa (`kill -9`), tunggu 10 detik, dan verifikasi bahwa layanan hidup kembali secara otomatis.
5. Bersihkan: nonaktifkan layanan dan hapus berkas unit setelah selesai.

Latihan 10.3 Investigasi Log dan Keamanan SSH

Analisis log sistem dan tingkatkan keamanan konfigurasi SSH.

1. Gunakan `journalctl -b -p err` untuk menemukan semua error sejak boot terakhir. Simpan hasilnya ke berkas dan hitung jumlah baris dengan `wc -l`.

2. Lakukan tiga perubahan keamanan pada `/etc/ssh/sshd_config`: tambahkan `PermitRootLogin no`, `MaxAuthTries 3`, dan `LoginGraceTime 30`. Ikuti alur aman: backup, edit, validasi `sshd -t`, reload.
3. Setelah reload, verifikasi tiga hal: layanan masih berjalan (`systemctl status ssh`), port masih mendengarkan (`ss -tlnp | grep ssh`), dan konfigurasi baru terbaca (`grep -E "PermitRoot|MaxAuth|GraceTime" /etc/ssh/sshd_config`).
4. Kembalikan konfigurasi SSH ke kondisi semula menggunakan berkas backup.

1.8 Referensi

- Freedesktop.org. *systemd System and Service Manager*. <https://www.freedesktop.org/software/systemd/man/latest/>
- Ubuntu Documentation Team. *Ubuntu Server Guide: Service Management*. <https://documentation.ubuntu.com/server/>
- Barrett, D.J., Silverman, R.E., Byrnes, R.G. *SSH, The Secure Shell: The Definitive Guide*, 2nd ed. O'Reilly Media, 2005.
- Nemeth, E., Snyder, G., Hein, T.R., Whaley, B. *UNIX and Linux System Administration Handbook*, 5th ed. Addison-Wesley, 2017.
- DigitalOcean. *Systemd Essentials: Working with Services, Units, and the Journal*. <https://www.digitalocean.com/community/tutorials/systemd-essentials-working-with-services>